



Computing Confidence Intervals for Coefficients in Logistic Regression [duplicate]

Asked 7 years, 1 month ago Modified 4 years, 5 months ago Viewed 52k times



7



This question already has answers here:

[Why is there a difference between manually calculating a logistic regression 95% confidence interval and using the confint\(\) function in R?](#) (3 answers)

Closed 7 years ago.



After fitting a logistic regression model in R using `model <- glm(y~x, family='binomial')` I can obtain the confidence intervals for the fitted coefficients using `confint(model)`, but I want to know how to manually compute these values. In the case of a linear model `lin_mod <- lm(y~x)` I can just do the following to obtain a 95% confidence interval for the slope coefficient:

```
CI_lower <- coefficients(lin_mod)[2] - 1.96*summary(lin_mod)$coefficients[2,2]
CI_upper <- coefficients(lin_mod)[2] + 1.96*summary(lin_mod)$coefficients[2,2]
```

Where `coefficients(lin_mod)[2]` is the estimated value of the coefficient, and `summary(lin_mod)$coefficients[2,2]` is corresponding standard error.

However when I use this same process to compute the confidence interval of the fitted coefficients of a logistic regression, the values don't agree with the output from `confint`. Below is an example using some randomly generated data:

```
x <- rnorm(n=100, mean=5, sd=2)
y_prob <- plogis(x, location=5, scale=1)
y <- sapply(y_prob, function(p) rbinom(1, 1, p))
model <- glm(y~x, family='binomial')

summary(model)$coefficients
#           Estimate Std. Error z value Pr(>|z|)
# (Intercept) -3.8998231  0.8838826 -4.412150 1.023490e-05
# x             0.7963213  0.1746632  4.559183 5.135303e-06

CI_lower <- coefficients(model)[2] - 1.96*summary(model)$coefficients[2,2] # =
0.4539815
CI_upper <- coefficients(model)[2] + 1.96*summary(model)$coefficients[2,2] # = 1.138661

confint(model)
#           2.5 %    97.5 %
# (Intercept) -5.8044657 -2.313925
# x             0.4843258  1.173998
```

As you can see, manually computing the 95% CI around the x-coefficient yielded `(0.4539815, 1.138661)` whereas computing it using `confint` yielded `(0.4843258, 1.173998)`. So my question is, how is `confint` computing this confidence interval, and why does my estimate differ? From some additional tests on larger samples I can see that the two estimates converge in the large-N limit, but I'm interested in what's going on for small N, in particular why the CI produced by `confint` is not symmetric about the coefficient estimate.

`r` `regression` `logistic` `confidence-interval` `regression-coefficients`

Share Cite Edit Follow Flag

asked Apr 23, 2017 at 23:07



jon_simon

2,049 2 17 29

1 Answer

Sorted by: Highest score (default) 



I just discovered that [someone answered this question in another post](#). The answer is, `confint` uses [profile confidence intervals](#), whereas I was computing a [Wald confidence interval](#) (which can equivalently be computed using `confint.default`).

14



[Share](#) [Cite](#) [Edit](#) [Follow](#) [Flag](#)

edited Dec 20, 2019 at 0:31

answered Apr 23, 2017 at 23:35

 [jon_simon](#)

2,049  2  17  29

